

Implementation Blueprint

Toolstack

Concern	Tool
Env / packaging	uv (lockfile-driven)
PDF rendering + metadata	PyMuPDF
Schema validation	Pydantic v2
CLI	Typer
Observability	Rich + JSON-Lines logs
Per-page language detection	lingua-py
Vector store	ChromaDB (embedded persistent)
Keyword index	SQLite FTS5
Text / caption embeddings	BAAI/bge-m3 via sentence-transformers (1024-d, multilingual)
Image embeddings	ViT-B-16-SigLIP via open-clip-torch (768-d, joint text-image space, L2-normalized)
AI access	Gemini API interaction layer (Pro tier for agents, Flash tier for callers)

Repository layout

We maintain a strict physical separation between the **Codebase Repository** (execution logic, CLI, static resources) and the **Project Workspace** (user data, corpora, derived databases).

Codebase Layout (**IndyRAG/**)

```
IndyRAG/
├── .gemini/          custom hooks and specialized agent skills
│   ├── hooks/       automated pre-commit verification scripts
│   └── skills/       specialized workspace skill definitions and instructions
├── docs/            design and architectural specifications
├── src/             codebase execution scripts grouped by concept
│   ├── hitl/        human-in-the-loop validation command modules
│   ├── orchestration/ pipeline orchestration, CLI command modules, and path scopes
│   ├── pdf_crop/    visual intelligence and cropping engines
│   ├── rag/         core RAG pipeline, indexing, and schemas
│   └── utils/       operational modalities, dynamic proxies, logging, and sandboxing
├── test_suite/      functional offline/live API test suite
├── .pytest_cache/   pytest caching folder and test run results
├── .venv/           uv managed virtual environment (gitignored)
├── .active_project  local pointer to the dynamically resolved active project (gitignored)
├── GEMINI.md        repository invariants and developer guidelines
├── GUIDE.md         user manual and command syntax reference tables
├── README.md        project overview, multi-agent diagrams, and governance report
├── pyproject.toml   project configurations and dependency list
└── uv.lock          exact pinned lockfile
```

Symmetrical workspace markers

Our multi-project resolution loop is anchored by a dual-marker directory sentinel handshake. This system establishes a bidirectional path-binding contract between the execution code repository and individual project workspaces while preserving absolute workspace portability and environment independence:

- **Active Project Pointer:** A local gitignored marker file (`.active_project`) is located at the root of the codebase repository. This file serves as a global fallback pointer, physically containing the absolute directory path of your preferred active project workspace. When executing commands directly from the codebase directory without any terminal environment variables or CWD context overrides, the orchestration layer reads this file to locate and bind securely to your target project folder.

- **Project Sentinel Marker:** A local empty marker file (`.rag_project`) is created at the root of every initialized project workspace. Symmetrically, this file acts as a lightweight, read-only directory sentinel. When climbing parent directories from your terminal's current working directory, the path resolver scans for this file to programmatically verify and validate the directory as a fully scaffolded, recognized project workspace before initiating any pipeline indexing, database writes, or visual-agent execution runs.

Multi-project concurrent resolution

Our system strictly isolates the codebase execution scripts from individual project directories. This allows multiple project workspaces to be processed simultaneously on the same machine.

The path resolution domain determines the active project workspace dynamically at runtime using three prioritized lookups:

1. **Environment Variable Override:** The system first checks for an environment variable named `RAG_PROJECT_ROOT` . If set, it binds immediately and exclusively to the specified folder path.
2. **Current Working Directory Climbing:** If no environment variable is present, the dynamic path resolution starts at your terminal's current working directory (CWD) and climbs up through the parent folders looking for a local project marker (the empty `.rag_project` file created during initialization).
3. **Global Fallback Pointer:** If neither of the above matches (such as when running commands directly from the codebase folder), the system reads the default project path written in the codebase's local active project file.

This three-tier resolution strategy guarantees that concurrent developer shells or automated batch runners operating in separate active project directories remain completely sandboxed, with zero cross-talk, database contention, or configuration collisions.

Model interaction layer

A dedicated layer handles all AI interactions, decoupling functional logic from the specifics of model communication.

- **Unified initialization.** Detection of the required environment and credentials occurs once, ensuring consistency across all automated and interactive steps.

- **Resilient error handling.** Communication errors are automatically caught and translated into typed connection failures carrying brief status codes (e.g., transport deadline exceeded, connection timeouts). Symmetrically, all user-facing console failure logs are run through a centralized Exception Briefer that programmatically extracts these concise codes (or maps local parsing errors into uppercase single-word briefs like `JSON_PARSE_ERROR`), completely avoiding verbose tracebacks and presenting a clean, professional command-line interface.
- **Tier governance.** Model versions and reasoning tiers are defined centrally. This ensures that agents and callers always use the appropriate level of reasoning without redundant configuration.
- **Pattern standardization.** Shared logic for one-shot calls and multi-turn sessions simplifies the application code and guarantees consistent processing of model responses.
- **Centralized Console Output Safety.** To prevent terminal formatting regressions resulting from third-party bracket-swallowing (where raw square bracket metadata is incorrectly parsed as styling markup and swallowed by the terminal), the command-line interfaces leverage a centralized console proxy. Symmetrically, this proxy intercepts all standard printing calls and routes them through a dedicated bracket-escaping sanitizer. Symmetrically, to maintain complete interface compatibility with advanced third-party progress-rendering modules, the proxy employs magic-method dynamic delegation, forwarding all unhandled console queries and context manager entries directly to the underlying standard terminal singleton.
- **Network Resilience Profile.** Delegated and structurally expanded inside its own dedicated architectural manual.
- **Synchronized parallel printing.** Symmetrically, to prevent overlapping stdout streams and terminal UI corruption when multiple concurrent threads attempt to print styled alerts simultaneously, the pipeline implements two distinct thread-safe printing architectures depending on the layer:
 - *Orchestrator/Agent Layer (Dynamic Callback Locking):* High-level orchestrators and visual agents receive a dynamically passed `echo` function callback parameter. To ensure safe, concurrent printing without hardcoding the output engine, they wrap and serialize execution calls on this passed handle using a local, thread-locked `echo_safe()` utility function.
 - *Low-Level Library Layer (Static Object Locking):* Decoupled rendering and coordinate processing utility functions do not accept dynamic callbacks in their signatures. Symmetrically, when reporting critical events (such as adaptive 40-megapixel downscaling warning alerts), they import the static, centralized console proxy directly and serialize stdout writes using dedicated, module-level static thread locks (e.g. `_CROP_CONSOLE_LOCK`).

Pipeline orchestration

The orchestrator detects current state from files and flags, runs the next automated step, and halts at every HITL gate.

Step sequence

1. Check in to a project workspace.
2. Environment audit & workspace directories scaffolding.
3. Seed inventory with documents.
4. Define the ontology (HITL Gate 1).
5. Approve document layers (HITL Gate 2).
6. Pass 1: Discover and draft structural Macro-Objects on disk.
7. Programmatic Macro-Object verification and catalog promotion.
8. Pass 2: Subdivide committed Macro-Objects into detail pairs on disk.
9. Programmatic Detail-Object verification and catalog promotion.
10. Approve regions (HITL Gate 3).
11. The captioner writes caption + vision prompt per region.
12. Mint URNs from approved regions.
13. The layer-assigner picks one layer per asset.
14. Approve assets (HITL Gate 4).
15. Build retrieval indexes.
16. Deep integrity verification.
17. Publish machine catalogs.
18. Ask questions against the indexed corpus (warmup).
19. Launch an interactive conversational REPL session.

Operational commands

The codebase exposes a robust CLI suite organized by conceptual roles in the project lifecycle:

- **Setup and Workspace Settlement:** Handles system environment pre-flight audits, dynamic path checks, and initializes isolated project directories.
- **Document Ingestion and Layering:** Scans the technical PDF corpus, performs multi-page language and format detection, and maps documents to ontology classifications.
- **Automated Drafting and Quality Verifications:** Launches multi-turn visual agents to discover regions, executes programmatic verifier-and-correction iterations, and manages the human-in-the-loop checkpoints.
- **Indexing and Retrieval Engines:** Compiles vector and keyword databases, routes queries across RRF-blended search channels, and drives Q&A synthesis and conversational REPL sessions.
- **Integrity Verification and Exporting:** Runs standalone deep-checks over the catalogs, hashes, and crop assets, and exports finalized approved bindings.

For the complete, end-to-end command mappings, CLI flags, and interactive execution examples, refer authoritatively to `GUIDE.md`.

Shared flags

- The visual-model cropping commands accept tier overrides for the agent tier only; the verifier remains configured on the smaller model by design.
- The three heavy commands (`macro-images` , `micro-images` , and `verify-drafts`) accept a `--deep` option to programmatically run up to 3 consecutive sweeping rounds, providing robust, self-healing retries and repairs against unrecoverable errors with early-exit optimizations.
- The retrieval commands accept a layer-affinity weight slider (default `0.3`) to blend vector hits by layer similarity.
- The retrieval commands accept a visual-dominant retrieval preset for visual-style questions.
- The retrieval commands support an opt-in query expansion flag that rewrites the question into synonymous variants spanning English and corpus source languages.

Region discovery

CAD drawings are vector PDFs with no embedded image objects. Semantically meaningful regions — callout balloons, connection annotations, spec blocks, exploded views — require visual understanding.

Strategy

- The discovery and subdivision agents open multi-turn conversation sessions, deciding from the coordinate grid overview whether to draft one or more regions.
- To prevent checklist hallucination, the drafting agents act as pure sprayers: they only propose crop coordinates using the drafting tool, completely omitting database write tools.
- A programmatic quality validation loop executes a synchronous verifier-and-corrections process up to three rounds, moving verified crops to the crops directories and committing to the regions database.
- All intermediate renders go strictly to a temporary scratch directory to prevent unverified leaks. Only successfully verified and promoted PNGs are moved to the production crops directory.

Ephemeral Zooms (Visual Context Pruning)

To prevent visual attention decay and context overflows during long-running multi-turn discovery sessions, the pipeline programmatically prunes intermediate visual payloads from the chat history. Symmetrically, after each tool interaction, the conversation history is parsed to strip out the massive base64 image data from all previous intermediate detail zooms, replacing them with lightweight text placeholders. To guarantee absolute visual grounding and prevent coordinate drift, the pipeline permanently preserves the foundational coordinate grid overview (the master visual anchor) and the most recent zoom crop (the active focus) completely intact. Symmetrically, this master-detail model maintains a highly compact token footprint while preserving maximum spatial reasoning capacity.

Sequential Execution & Programmatic Census Validation

To enforce strict state consistency and prevent context bloat, the pipeline implements hard programmatic safeguards inside the tool execution and validation layers: – **Sequential Turn-Throttling (Sequential Discipline)**: If an agent attempts to execute multiple crop tool

calls in parallel within a single turn, the system programmatically intercepts the subsequent calls and raises a sequential discipline violation. This prevents conversational turn accumulation, context-window overflow, and subsequent connection failures. -

Programmatic Label Matching: Symmetrically, when a crop is drafted, the callback layer loads the page's registered on-disk census checklist and cross-references the requested label. If the label does not exist on the checklist, the call is rejected with a census validation violation. This enforces strict naming alignment and prevents orphaned pending states inside our checklist files on disk. -

Turn-Limit Exhaustion Error (Loop Protection): Symmetrically, if an agent exhausts its maximum allowed turns without explicitly declaring task completeness or breaking the loop, the low-level runner programmatically raises an abort exception. This upgrades silent, incomplete exits into an explicit, red-styled system failure, immediately triggering the standard closed-loop rollback and checklist reversion routines to preserve database and spatial coordinate integrity.

Programmatic Anti-Hallucination Techniques

To protect against LLM hallucination and ensure absolute naming, spatial, and completeness alignment during agentic execution loops, the pipeline enforces strict programmatic guards: - **Strict Label Cross-Referencing:** When agents call the crop tool, the system programmatically cross-references the requested label against the pre-registered on-disk census checklist. If the agent hallucinates a custom synonym instead of the exact registered label, the tool call is instantly rejected with a census validation error (preventing un-registered or orphaned annotations). - **Sequential Turn-Throttling (Sequential Discipline):** If an agent attempts to execute multiple crop tool calls in parallel within a single turn, the system programmatically intercepts the subsequent calls and raises a sequential discipline error. This prevents conversational turn accumulation, context-window overflow, and subsequent connection failures. - **Local-to-Global Coordinate Interpolation:** Symmetrically, the subdivision engine utilizes programmatic interpolation functions to mathematically translate local coordinate boxes (drawn by a micro-agent over a zoomed crop) back into absolute global page coordinates, preventing out-of-bounds global coordinate hallucinations. - **Turn-Limit Loop Protection:** If an agent exhausts its maximum allowed conversational turns without declaring task completeness or breaking the loop, the runner programmatically raises an agent abort exception. This halts infinite correction loops and "faked completeness" hallucinations, forcing an immediate rollback to preserve database integrity. - **Anti-Faking Completeness Verification:** The execution loops enforce a strict sequence discipline. The orchestrator tracks the expected volume of elements registered in the initial plan and compares it against successful tool executions. If an agent hallucinates a "task complete" state while items remain pending, the exit is forcefully blocked with a completeness violation error.

Quality enforcement

- The image-quality rules (parsed dynamically by section from the rules definition file) are injected verbatim into the agent's instructions.
- To prevent rule drift during long sessions, the rules are also appended after every crop result and re-stated as a final reminder.
- The verifier (Flash tier, one shot per crop) re-checks every crop against the same rules. Failures are handled by the refinement session.

Judge-Producer asymmetry

Discovery and refinement run on the Pro tier for multi-turn visual reasoning. Verification runs on the Flash tier on purpose: a weaker judge over a stronger producer prevents the producer from rationalizing its own output.

Parallel Verification Loop Performance Optimization

Symmetrically, executing verifications and corrections concurrently within a parallel thread pool presents a critical disk-I/O bottleneck: if each worker thread synchronously loads and parses the catalog inventory YAML from disk to resolve document paths, the resulting lock contention and file-I/O thrashing severely degrade performance. Symmetrically, to achieve peak software craftsmanship and eliminate this latency, the pipeline pre-resolves and caches all absolute corpus document paths exactly once on the main thread before starting the verification pool. Symmetrically, this pre-resolved cache is passed down to the worker execution frames for instantaneous, zero-I/O memory lookups. To guarantee absolute robustness and allow sandbox-isolated functional tests to bypass the loading phase cleanly, the caching engine implements an elegant try-except fallback that dynamically loads paths on-the-fly only if the required identifier is missing from the pre-resolved cache. Furthermore, to prevent developer errors, namespace shadowing, and namespace leakage across active loops, all transient API error and connection classification helpers are consolidated strictly under their true logical resilience domain, completely purging redundant shadow imports and re-exports from any other system modules.

Macro-Level Discovery Fast-Track vs Micro-Level Subdivision State Enforcement

The skip operation is handled differently during macro-level discovery and micro-level subdivision steps to maximize pipeline throughput and speed up the parsing of pure prose pages in text documents: - **Macro-Level Discovery Fast-Track:** In technical manuals,

datasheets, or drawings, many pages consist entirely of prose paragraphs (narrative instruction text, license disclosures, or Table of Contents). The discovery step can immediately call the skip operation on these pages. It does not enforce any planning or execution warnings on skipped pages, but simply records the skip completion. Symmetrically, the programmatic verifier implements a fast-track rejection on these skipped pages, immediately finalizing them as unrecoverable exclusions without ever invoking the multi-turn re-cropping agent. This allows the pipeline to rapidly parse and exclude vast lots of pure prose text pages at lightning speed while completely avoiding API context window token limit overflows. – **Micro-Level Subdivision State Enforcement:** In contrast, the subdivision step operates under a strict, state-gated sequence within already promoted macro views. Since the parent macro object has already been validated as having high catalog value, skipping individual elements requires granular task tracking and is subjected to sequence and exit enforcers. To permanently capture and persist this skip state inside our centralized, single-source-of-truth census database, the skip macro view operation programmatically writes an empty checklist partition file to disk. Symmetrically, this explicitly serializes a completed checklist, allowing subsequent run invocations to dynamically query the census partition and instantly skip the macro-view without requiring any redundant, duplicate state flags inside the regions database. – **Divergent Completed-with-Warnings Logic:** Because skipped pages write absolutely no census database records under the fast-track approach, the completed-with-warnings metric checker implements two fundamentally different calculation paths to prevent status reporting drift: (1) During the macro discovery phase, the engine strictly counts only the unresolved, pending items located within existing, registered page-census database files on disk, completely ignoring completely unprocessed or fast-track skipped pages (which lack on-disk census files). (2) In contrast, during the micro-subdivision phase (where skipped parent macro-views explicitly serialize empty checklist files to disk), any missing census record strictly represents an un-processed or interrupted view, allowing the engine to safely count both registered pending items and un-visited macro-views, maintaining absolute metric-reporting accuracy!

Captioning

For every approved region, the model produces two fields in a single call with the crop image:

- **vision_prompt** — a paragraph describing what is in the crop, written first.
- **caption** — a short semantic label (≤ 1 sentence), written second.

The rules bias the model toward the main subject of the crop and away from peripheral text near the borders. Captioning always overwrites existing values.

URNs and identifiers

Format

```
asset://<machine>/<doc_id>/IMG-<label>
```

- `<machine>` — the machine ID (corpus folder name).
- `<doc_id>` — the document identifier, prefixed with the machine ID.
- `IMG-<label>` — fixed kind prefix plus a hyphenated slug of the label.

The **layer** is **not encoded in the URN**. URNs are provenance identifiers, not classification keys.

Filesystem slug

A deterministic mapping turns a URN into a safe filename by replacing non-alphanumeric characters with underscores.

Work example:

```
asset:///3048/3048-CAD-159825-ZUSAMMENBAU/IMG-front-elevation  
→ asset__3048_3048_CAD_159825_ZUSAMMENBAU_IMG_front_elevation
```

Indexing

Five indexes are built in one pass from the inventory and the corpus.

Index	Backend	Unit	Embedder
<code>text_chunks</code>	ChromaDB + FTS5	one record per (page, section) group of merged blocks	<code>bge-m3</code>

Index	Backend	Unit	Embedder
<code>captions</code>	ChromaDB	one record per asset (caption + vision prompt combined)	<code>bge-m3</code>
<code>images</code>	ChromaDB	one record per crop PNG	<code>SigLIP</code>
<code>id_search</code>	FTS5	one record per asset (display name + caption, deduped)	—
<code>layer_index</code>	ChromaDB	one record per layer (descriptor: <code>name + description + keywords</code>)	<code>bge-m3</code>

Design choices

- **Text-rich documents only.** `text_chunks` skips CAD drawings; their content is captured by the captions and images channels.
- **Prose only.** Table-cell shapes and numeric-heavy blocks are dropped from the text search index.
- **Cross-language retrieval.** Non-English page blocks are translated once at build time. The translation is **section-aware**: blocks are sent with their enclosing section heading to ensure technical terminology resolves correctly.
- **Section-aware chunking.** Blocks on a page are merged by their section heading. Lists and procedures stay together as coherent semantic units.
- **No layer prefix in FTS5.** Layer routing lives entirely on the vector side via `layer_index` .
- **Approval required.** Every document and asset must be HITL-approved before indexing.
- **Hybrid Parallel Concurrency.** Document indexing decouples network-bound asynchronous translations (MIMD) from local compute-bound vectorizations (SIMD). Background threads translate pages concurrently globally to saturate network APIs, while local embeddings are computed sequentially per document in safe batches of 64 (`EMBED_BATCH_SIZE`) inside global locks to protect macOS unified memory.

Operational guarantees

Structural refusals

- URN minting is refused while any document is unapproved.

- Index building is refused unless every document and asset is approved.

Exit codes

Code	Meaning
0	Success
1	General error
2	Verification check failed
3	Gemini API preflight failed (network or auth)
4	Document layer unverified
5	Environment audit incomplete
6	Inventory parse error
7	Index build blocked

Data integrity rules

To protect the consistency of catalogs and prevent on-disk corruption under concurrent execution, the system enforces the following transactional boundaries:

- **Atomic Serialization:** All catalog metadata writes are staged in temporary files and renamed atomically to prevent partial write failures.
- **Visual Scratch Staging:** Proposed visual crops are written to a temporary scratch folder and promoted to the canonical assets library only on explicit verifier-led commits.
- **Resolution Downscaling:** Renders are capped at forty megapixels, and oversized visual crops are dynamically downscaled in memory before transport to ensure model parsing safety.

- **Terminal Output Hygiene:** Low-level document parsing and formatting warnings are suppressed natively and thread-safely at the library engine level. During document ingestion, all intercepted anomalies are parsed and atomically cataloged to an isolated formatting violations manifest on disk, allowing operators to track legacy layout issues proactively. Symmetrically, any on-screen warnings are triggered lazily only when a true violation is parsed, keeping the terminal completely silent and pristine for compliant, legacy-free corpora.
- **Concurrent Embedding Compilation:** Vector generation and translations are processed concurrently across thread pools to exploit multi-core scaling, utilizing memory-buffered chunk boundaries to write elements in unified transactions.

Developer Integration, Version Control, and Automated Hooks

The codebase includes robust integration policies to ensure long-term stability and coordinate multiple concurrent developers:

- **Strict Git Source Isolation:** All raw corpus PDFs, ChromaDB databases, SQLite full-text tables, and JSON-Lines log files are strictly gitignored, keeping our code repository lightweight and securely isolated from project-specific data.
- **Automated Syntax Control Hook:** To prevent syntax drifts or parameter typos from entering our codebase, we enforce a strict multi-tier pre-commit hook. Whenever a developer modifies more than 10 lines of Python code, the hook automatically triggers three sequential passes:
 - *Pass 1:* Grammatical syntax and compile verification.
 - *Pass 2:* Static AST analysis checking call signatures and schema attribute types.
 - *Pass 3:* Comprehensive execution of our complete 207 offline functional test cases, passing 100% green before any code can be committed.
- **Continuous Un-Mocked Live Pipeline Testing:** Visual intelligence and integration endpoints are strictly decoupled to prevent latency during standard local development. When explicitly invoked, the framework executes a continuous, un-mocked visual pipeline test natively against the Gemini API (validating discovery, simulated quality failures, and bounding-box corrections in sequential continuity).